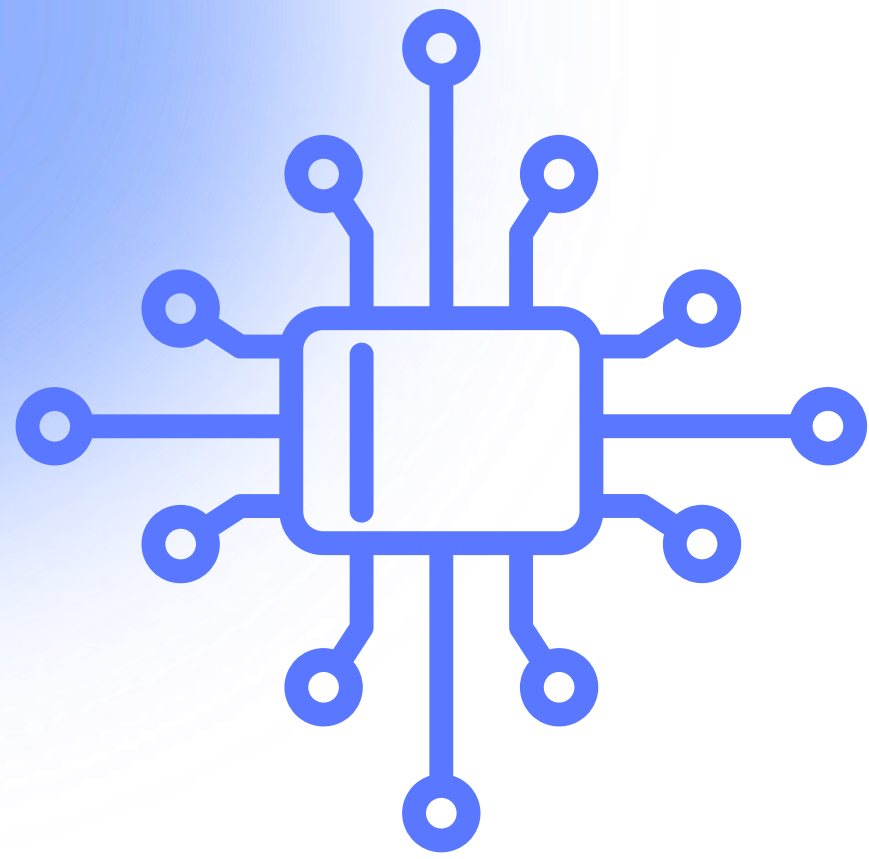
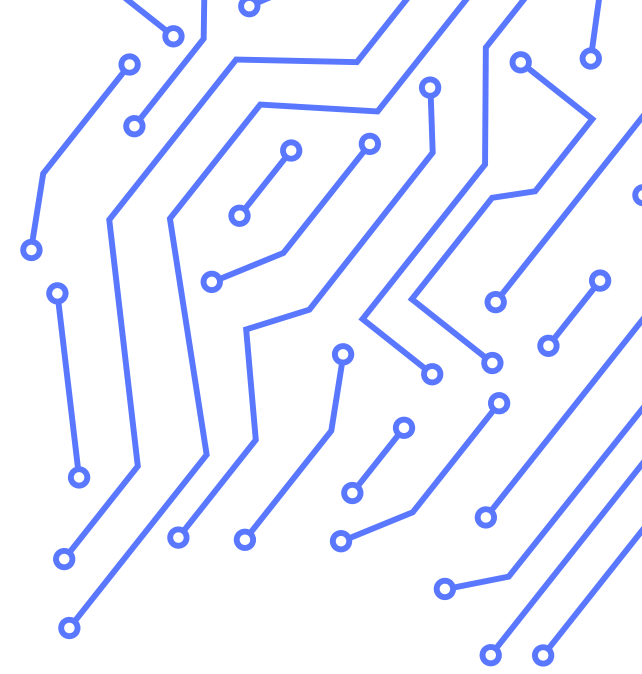
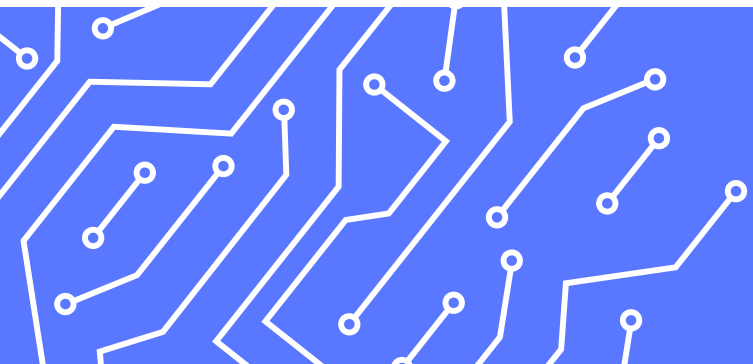




OPTIMIZERS & TRAINING PROCESS



**MADE BY:
HAZEM GAMAL**

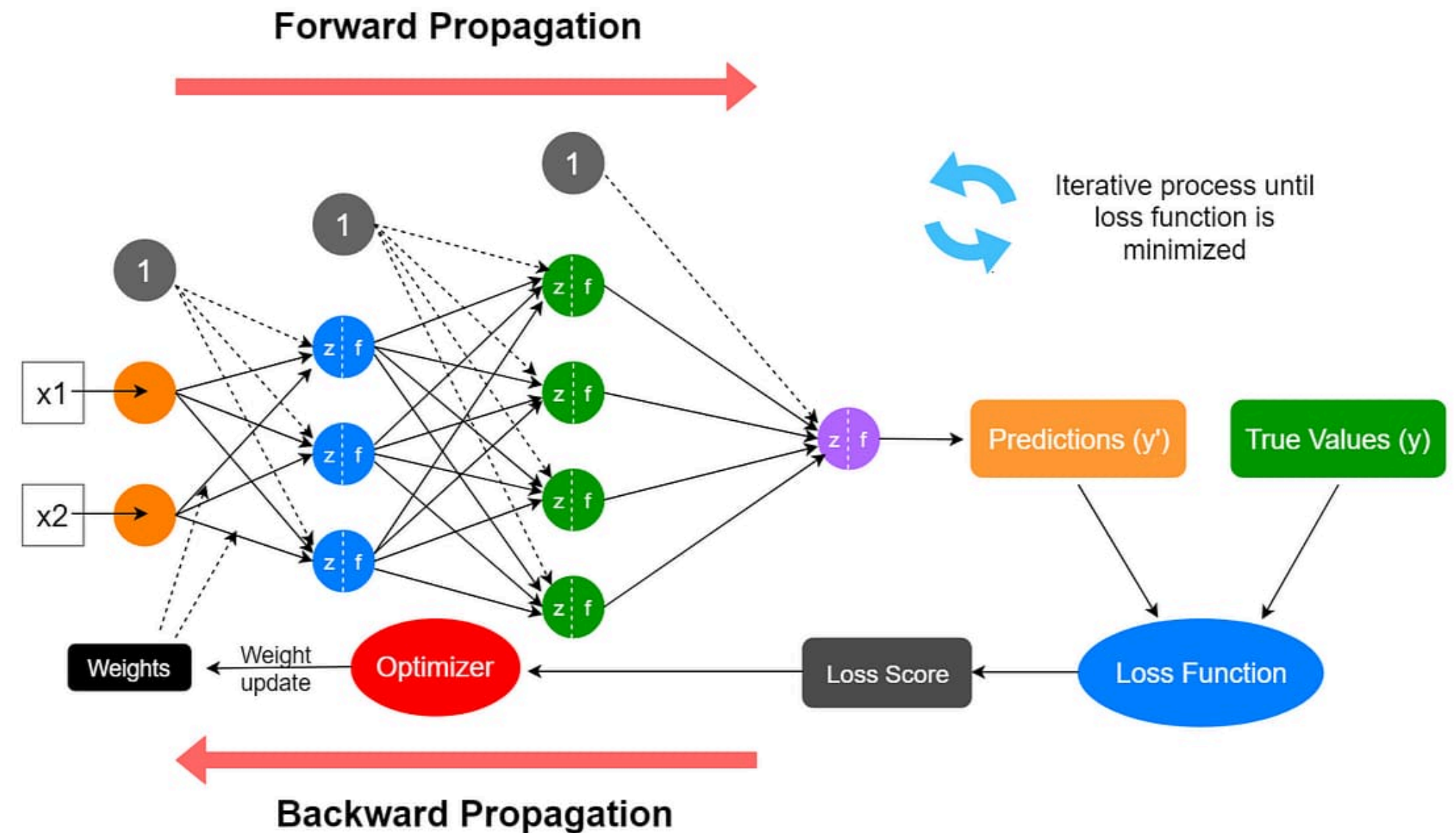


OPTIMIZERS

The mechanism that turns errors into learning.

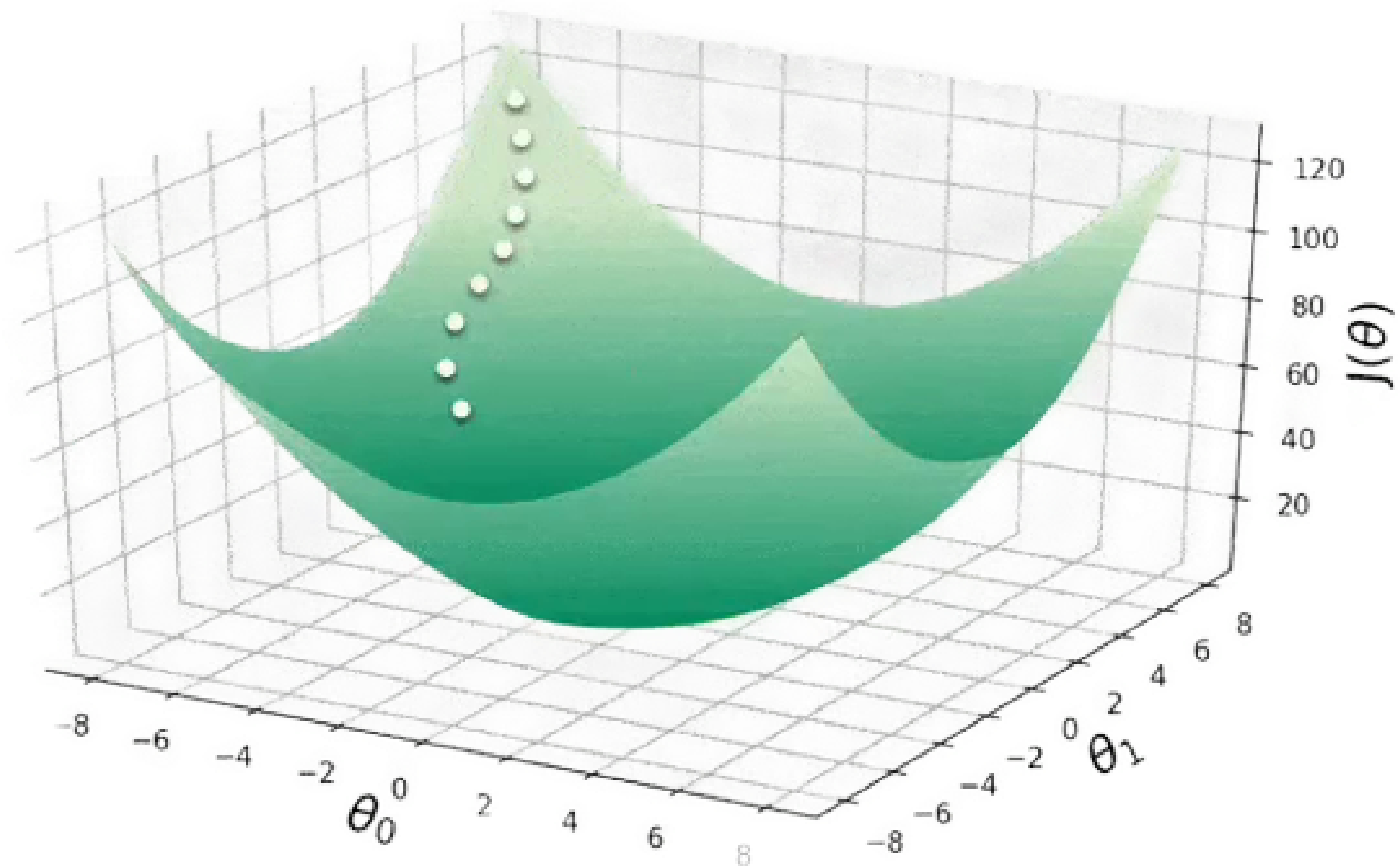
Key Points:

- Networks learn by adjusting weights, not by "**memorizing answers**"
- The **optimizer connects**: Weights $\leftrightarrow$ Loss $\leftrightarrow$ Gradients
- **Goal**: minimize the loss function over time



GRADIENT DESCENT

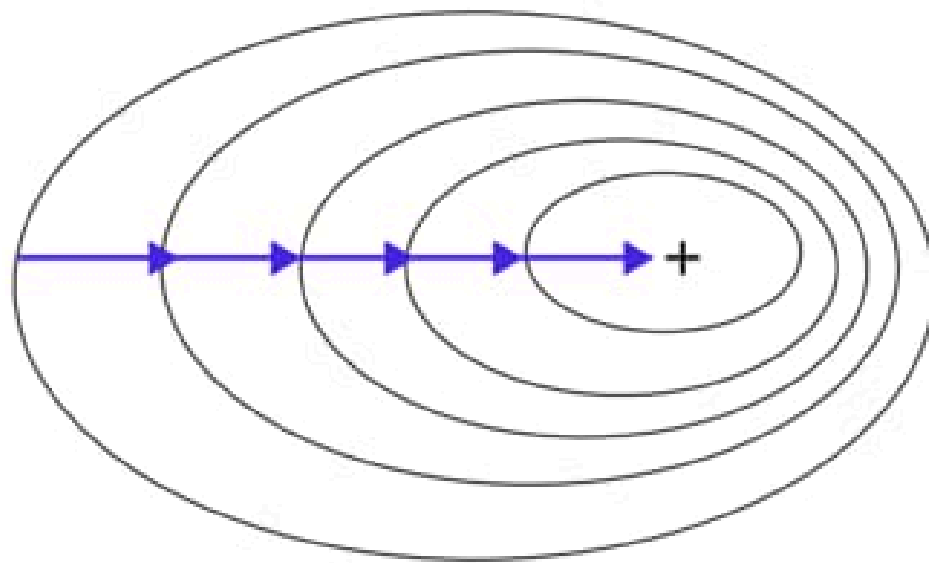
Finding the lowest point of the loss "landscape".



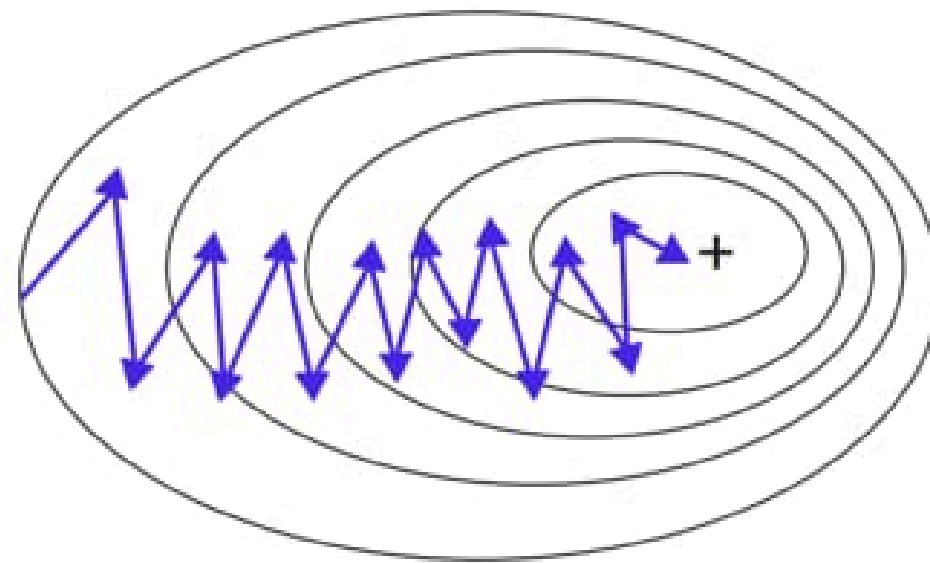
TYPES OF GRADIENT DESCENT

Batch GD vs SGD vs Mini-Batch GD

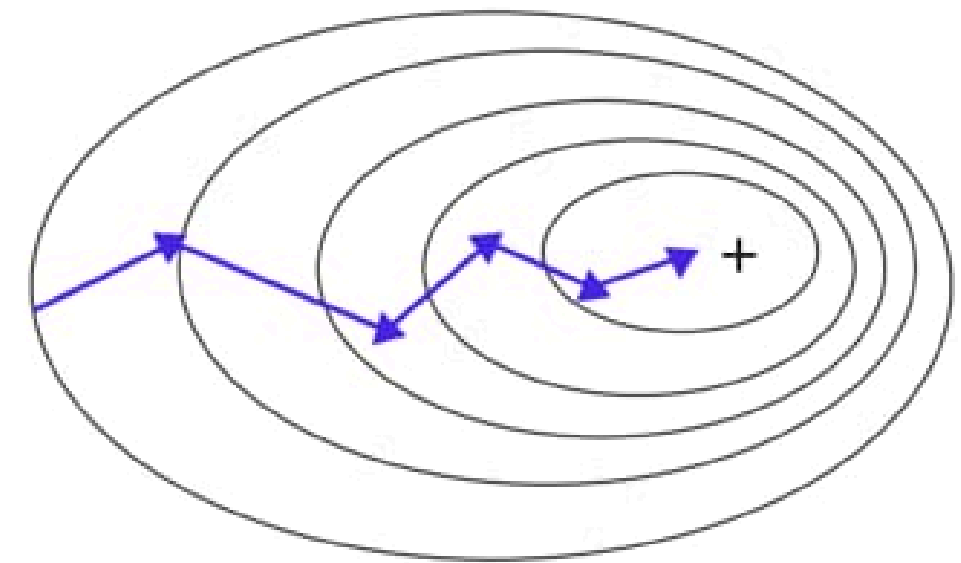
Batch Gradient Descent



Stochastic Gradient Descent



Mini-Batch Gradient Descent



Criteria	Batch GD	SGD	Mini Batch GD
Samples/update	All	1	Small group
Speed per update	Slow	Fast	Medium
Memory usage	High	Low	Medium



TYPES OF OPTIMIZERS

SGD Optimizer

Momentum

RMSprop

Adam



SGD OPTIMIZER

The foundational, simplest optimizer

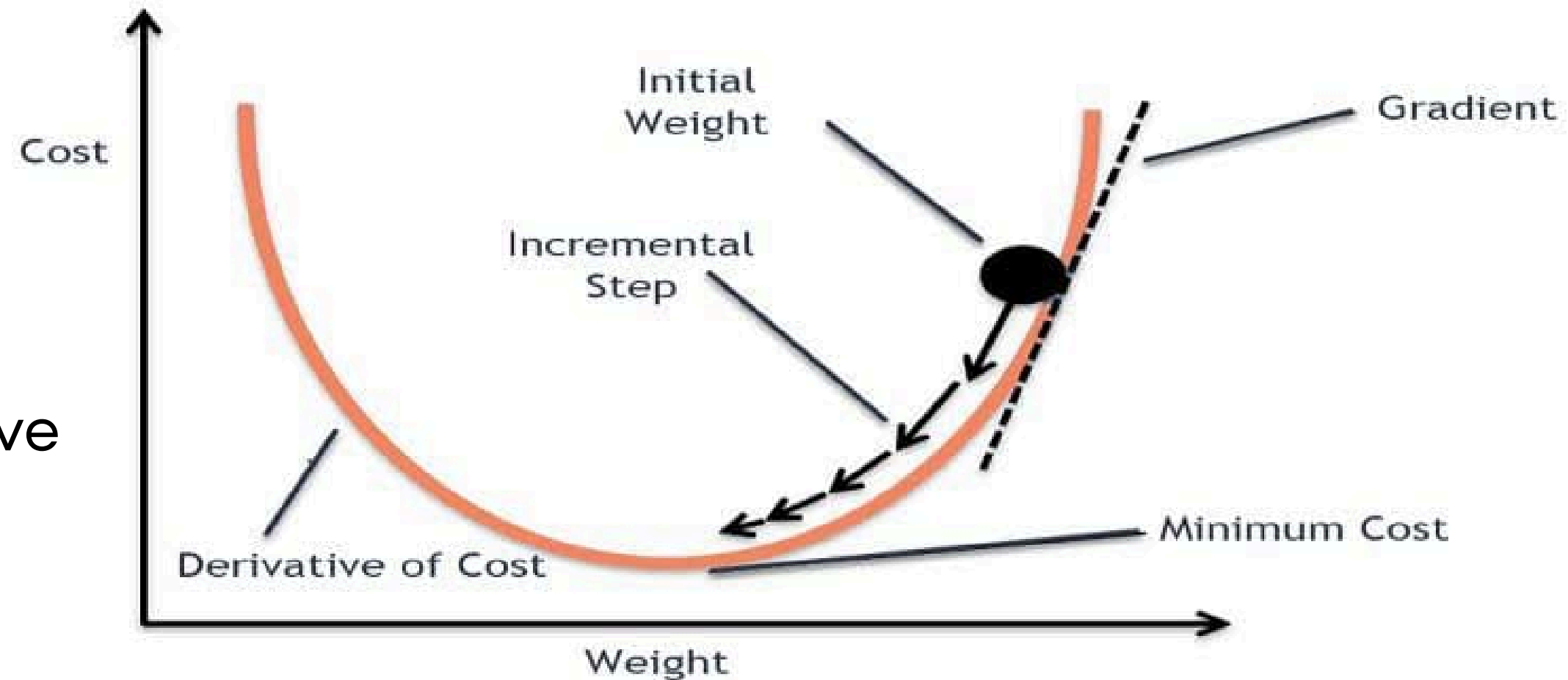
Key Points:

- Basic weight update using gradients
- Same learning rate for all parameters
- **Limitation:** no adaptivity, sensitive to noisy gradients

Equation:

$$w_{t+1} = w_t - \eta g_t$$

- w_t = current weight
- $g_t = \nabla L(w_t)$ = current gradient
- η = learning rate



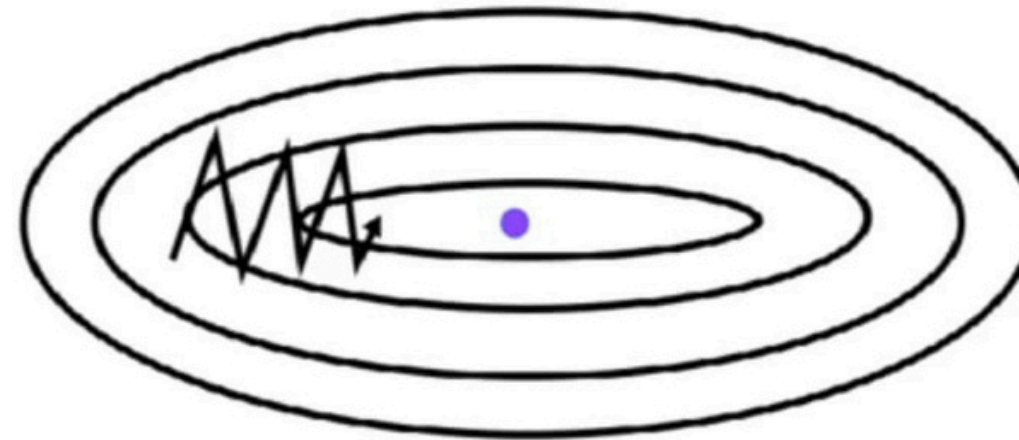
MOMENTUM

Giving the optimizer some "inertia"

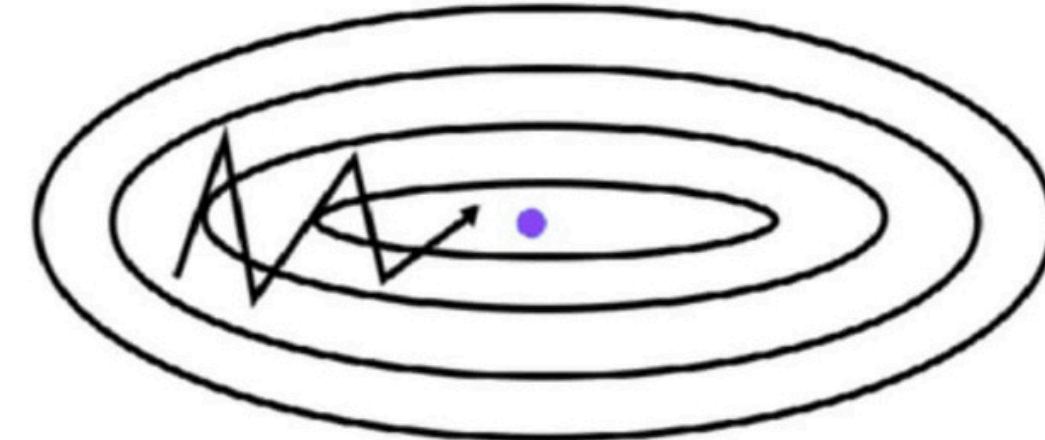
Key Points:

- **Problem:** plain SGD can be slow / oscillate
- **Solution:** accumulate previous gradients
- **Result:** faster, smoother movement in consistent directions

SGD without momentum



SGD with momentum

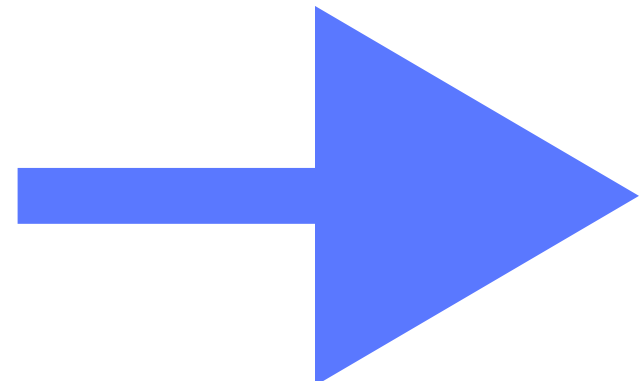


Equation:

$$v_t = \beta v_{t-1} + g_t$$

Then update the weights:

$$w_{t+1} = w_t - \eta v_t$$



- v_t = velocity
- β = momentum coefficient
- g_t = current gradient
- η = learning rate

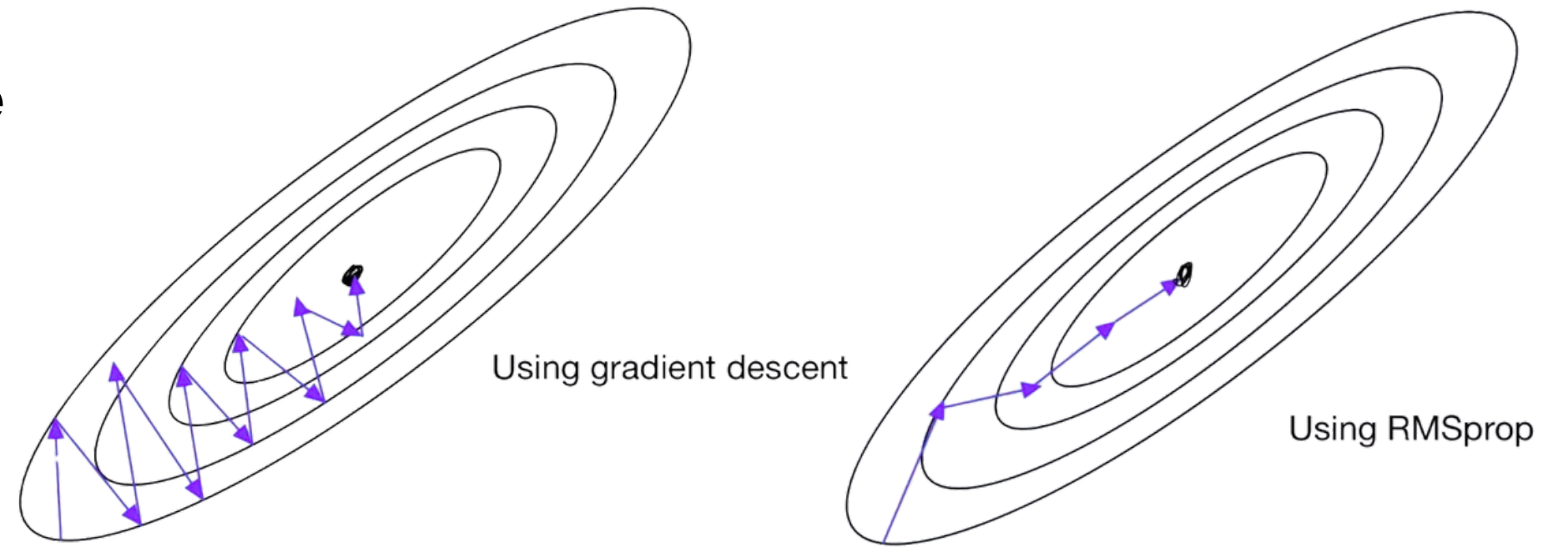


RMSPROP

Giving each parameter its own learning rate

Key Points:

- **Motivation:** one fixed learning rate isn't optimal for all weights
- Adaptive, per-parameter learning rates
- Helps stabilize training on uneven loss surfaces



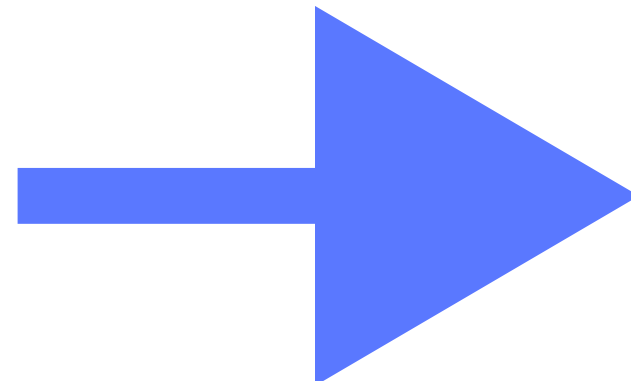
Equation:

Step 1 — Keep a moving average of squared gradients

$$s_t = \beta s_{t-1} + (1 - \beta) g_t^2$$

Step 2 — Update the weight

$$w_{t+1} = w_t - \frac{\eta}{\sqrt{s_t} + \epsilon} g_t$$



- s_t = moving average of squared gradients
- g_t = current gradient
- η = learning rate
- β = decay rate
- ϵ = small number to avoid division by zero



ADAM

Combination of Momentum and RMSprop

Equation:

Adam keeps **two moving averages**.

First moment – Momentum-like

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

This tracks the **average direction of the gradients**.

Second moment – RMSprop-like

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

Final update

$$w_{t+1} = w_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

Typical values:

$$\beta_1 = 0.9, \quad \beta_2 = 0.999$$



EPOCHS VS BATCHES VS ITERATIONS

Batch

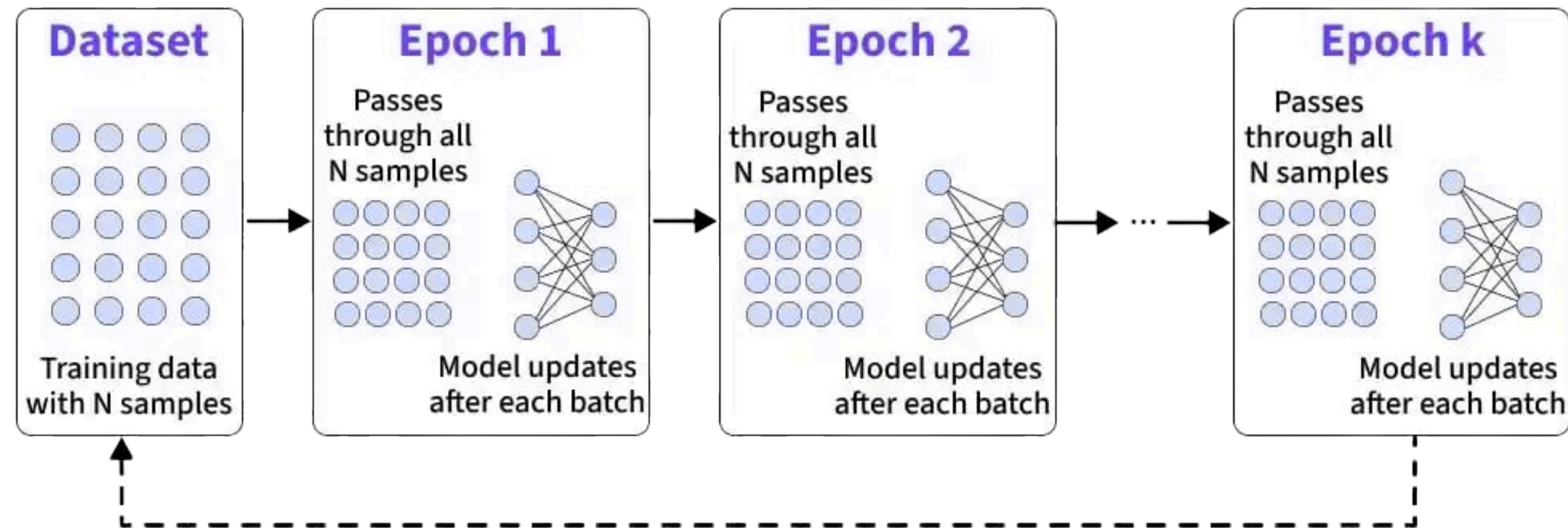
- Splitting data into manageable chunks

Iteration

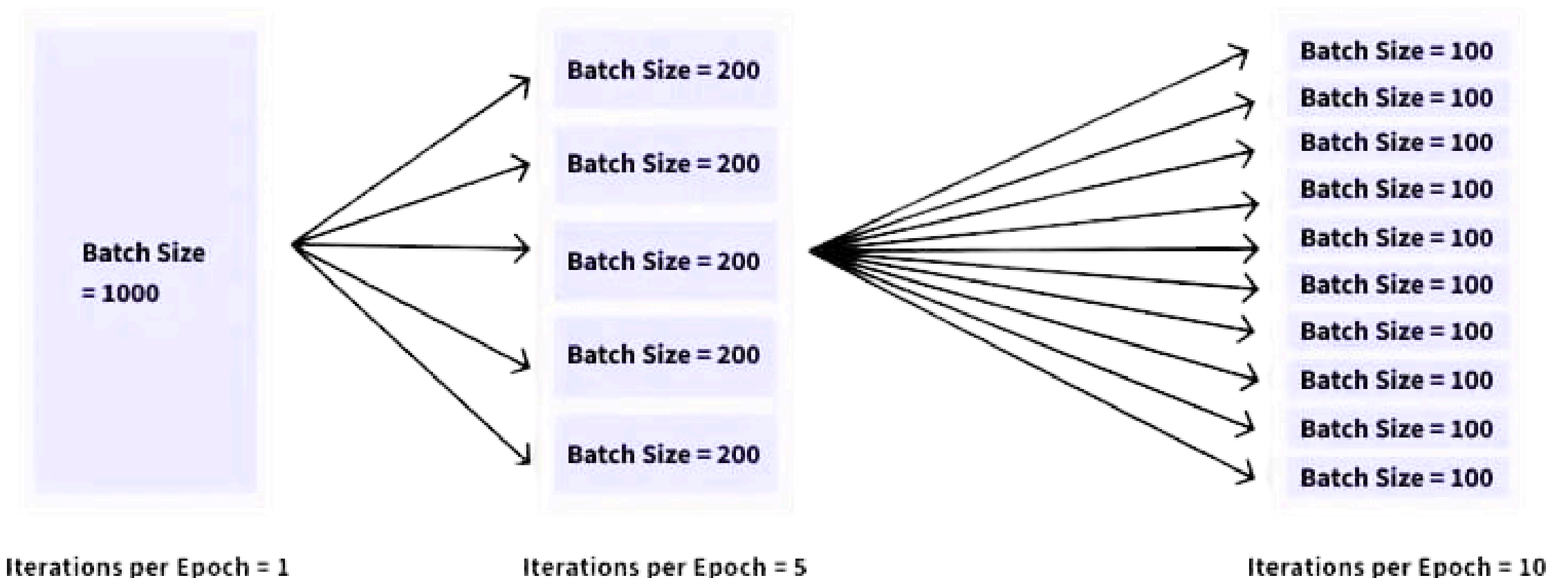
- One pass, one update.

Epoch

- One full trip through the entire dataset



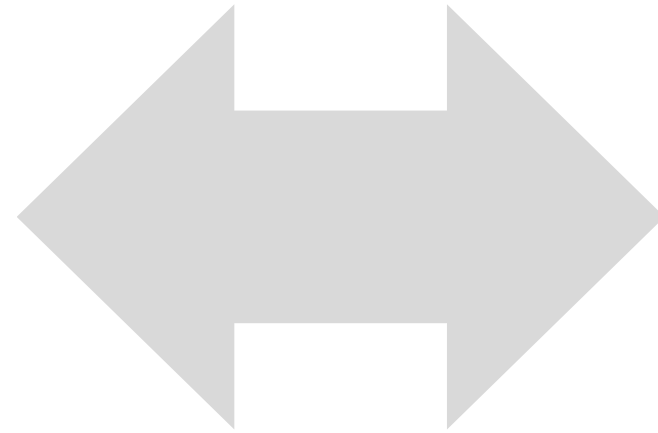
Repeat for next epoch until the model converges or for a fixed number of epochs



TRAINING PROCESS

Training Loop

1. Initialize the model
2. Load training data
3. Forward Pass
4. Calculate Loss
5. Backpropagation
6. Calculate Gradients
7. Update Weights
8. Repeat for all batches
9. Repeat for multiple epochs
10. Evaluate the Model



```
import torch
import torch.nn as nn

# Model
model = nn.Linear(2, 1)

# Loss function
loss_function = nn.MSELoss()

# Optimizer
optimizer =
torch.optim.SGD(model.parameters(
), lr=0.01)

# Training loop
for epoch in range(10):

    # 1. Forward pass
    prediction = model(X)

    # 2. Calculate loss
    loss =
loss_function(prediction, y)

    # 3. Backpropagation
    loss.backward()

    # 4. Update weights
    optimizer.step()

    # Clear gradients
    optimizer.zero_grad()
```



THANK YOU

I hope you understand the
session well

Thanks for your attention